MQL5.community
Follow us on socials
Be the first to know
>>



Нейронная сеть на практике: Вопрос шкалы

0 Daniel Jose 76

Введение

В предыдущей статье «Нейронная сеть на практике: перцептрон» было более полно и подробно объяснено всё, что мы создали до сих пор. В основном это был обзор, призванный показать, как всё возникает естественным образом по мере того, как математика подводит нас к тому, что нам действительно нужно реализовать.

В этой статье мы рассмотрим и изучим, зачем нужно менять шкалу данных, которые будут использоваться. Для большинства людей цель этого изменения остаётся в некотором смысле неясной, потому что многие делают это, не до конца понимая, зачем это нужно, а другие просто считают, что делать этого не нужно.

Если не понимать как следует, зачем вносить или не вносить это изменение в данные, довольно сложно понять, как сеть перцептронов может давать те результаты, которые мы обычно видим.

Зачем менять шкалу данных?

В этой статье мы не будем уделять столько внимания коду и больше не будем говорить о методе наименьших квадратов или упоминать его, кроме очень конкретных случаев. Хотя он и останется в коде класса C_Neuron, он будет там только потому, что нам нужно будет объяснить ещё один вопрос, которого мы пока не касались и в котором как раз используется функция потерь метода наименьших квадратов. С этого момента основное внимание будет уделяться использованию градиента. Как бы то ни было, я не хочу, чтобы вы слишком привязывались к какой-либо части кода. Я хочу, чтобы вы постарались понять, почему стоит использовать одно или другое и когда это делать.

Причина очень проста: когда речь идёт о сетях перцептронов, не существует одного идеального способа всё делать. Есть то, что работает, и то, что не работает. При этом во многих случаях эффективный подход может оказаться довольно ресурсоёмким с точки зрения вычислений, поэтому его нужно будет адаптировать к более подходящей модели. Для этого вам нужно знать, как изменить код. По мере того как мы будем добавлять всё больше входов в перцептрон и увеличивать количество перцептронов, эти вопросы будут становиться всё более понятными, и вы лучше поймете их на практике.

Среди широкой публики бытует ложное представление о том, что сеть перцептронов или искусственный интеллект могут в некотором смысле понимать мир, в котором мы живём. В некоторых случаях это даже может быть правдой. Но вам, если вы хотите добиться как можно более быстрой сходимости сети перцептронов или даже отдельного перцептрона, нужно понимать, что шкала значений часто в итоге влияет на скорость сходимости.

В каком-то смысле это может показаться ерундой и даже чем-то противоречащим интуиции, поскольку во время обратного распространения ошибки кажется, что перцептрон сначала делает относительно большие скачки в сторону точки сходимости. А со временем мы начинаем делать всё более маленькие

шаги, исправляя эту ошибку. Так что на первый взгляд изменение шкалы данных не должно оказывать слишком большого влияния. Да, это верно во многих случаях. Тем не менее существуют вполне конкретные ситуации, в которых изменение шкалы значений действительно может заметно повысить скорость сходимости. Кроме того, есть ещё один фактор, из-за которого изменение шкалы становится довольно интересным. Этот фактор связан с вопросом, который я объясню, когда мы будем говорить о нормализации и стандартизации. Но, немного забегая вперёд, это было бы похоже на то, как если бы мы в некоторой степени уменьшили смещение при обучении. Зачастую сеть или даже отдельный перцептрон в конечном итоге приобретают определённое смещение именно потому, что работают с необработанными данными.

И снова: не всегда нужно делать то, что будет показано здесь. На самом деле я пока не уверен, оставлю ли я то, что мы здесь увидим, в каком-нибудь будущем коде. Поскольку я пока не принял решения по этому поводу, вам нужно понимать, что в зависимости от назначения вашей сети перцептронов или отдельного перцептрона вам придётся каким-то образом обработать данные ещё до того, как они пройдут через перцептрон или сеть перцептронов. Это происходит на этапе обучения. После завершения обучения обработка данных может больше не понадобиться. Но, опять же, такую ситуацию нужно разбирать спокойно.

Один из самых простых способов обработки заключается в изменении шкалы значений. Чтобы понять важность такой обработки, я хочу, чтобы вы нашли на финансовом рынке инструмент, по которому выплачиваются дивиденды. Если вы посмотрите на исторический график этого же инструмента, то заметите, что размер баров постепенно уменьшается, так же как и котировка этого инструмента. Это создаёт у нас ложное впечатление, что процентное изменение со временем изменилось, хотя котировка этого инструмента на протяжении всего периода оставалась примерно на одном и том же уровне. Наше поверхностное восприятие подскажет нам прямо противоположное тому, что происходит на самом деле. И подобные ситуации в конечном итоге создают у нас смещение.

То же самое происходит во время обучения сети перцептронов или отдельного перцептрона. Если данные представлены определённым образом, то в результате вычислений получится полностью смещённое уравнение. И это происходит именно потому, что система вынуждена использовать довольно разные значения, хотя на самом деле они находятся в одном и том же диапазоне, а это показывает, что инструмент не демонстрирует того движения, которое, как нам кажется, мы наблюдаем. В результате система вынуждена выдавать ложноположительный или ложноотрицательный результат.

Пока что я лишь начинаю эту тему, так как она будет рассмотрена более подробно, когда мы будем говорить о нормализации и стандартизации. Там вы действительно поймёте, насколько эта обработка может повлиять на результат: она может очень помочь, но и серьёзно навредить, если не принять все необходимые меры предосторожности, чтобы избежать разброса данных. Здесь же пока что эту идею понять гораздо проще, и она послужит лишь основой для чего-то гораздо более глубокого.

Чтобы вы могли лучше это понять, давайте представим себе набор данных, значения которого распределены в диапазоне от нуля до пятнадцати тысяч. Для перцептрона или даже сети перцептронов это распределение не имеет большого значения, поскольку в конечном итоге внутренние переменные, которые мы называем весами и смещением, будут скорректированы так, чтобы сформировать уравнение, отражающее разброс этих данных. Таким образом, даже если мы изменим шкалу так, чтобы значения лежали в диапазоне от нуля до единицы, в принципе это не даст большой разницы, если мы сохраним то же распределение.

Хотя изменение шкалы и не меняет это распределение, оно всё же может сделать факторизацию немного более эффективной. Помните, что мы начинаем с внутренних значений в диапазоне от нуля до единицы, заданных псевдослучайным образом. Хотя перцептрон, используя градиент, и пытается быстро проанализировать блок данных и сойтись к какому-то решению, вначале мы можем получить огромную ошибку, которая будет постепенно уменьшаться. Однако если мы начнём с другой шкалы, эта ошибка может оказаться значительно меньше или больше, а может быть даже настолько малой, что приблизится к целевой (ожидаемой) ошибке.

Таким образом, когда мы снова увеличим шкалу, перцептрон или сеть уже не будут настолько смещены, поскольку между вычислениями, выполняемыми перцептроном или сетью, и ожидаемыми значениями будет больший запас ошибки. И если у нас есть ожидаемые значения, очень близкие друг к другу, перцептрон или сеть не будут точно знать, какое из них является правильным, что даст нам возможность понять, какое значение было бы наиболее подходящим. Это связано с тем, что перцептрон или сеть могли бы указать нам, какие значения ближе всего к правильному. И этого не происходит, когда сеть или перцептрон уже в некотором роде смещены под влиянием определённого набора данных.

Так что здесь мы сосредоточимся именно на вопросе шкалы. Мы сделаем это ещё до того, как попытаемся понять, зачем нужно нормализовать или стандартизировать значения в базе данных. В некотором смысле стандартизация или даже нормализация данных — это всего лишь приведение разброса значений к определённому диапазону. Хотя на первый взгляд это может казаться разными вещами, основная цель у них одна и та же. Так что, чтобы начать рассматривать это в виде кода, давайте откроем новую тему и чётче разграничим всё это.

Разница между шкалами

Первое, о чём нам нужно подумать, напрямую связано с математикой. Я хочу, чтобы вы сначала это поняли и не думали, будто мы делаем одно, тогда как на самом деле делаем другое.

По сути, есть два довольно известных типа шкал. Одна — арифметическая шкала, другая — логарифмическая. Но какую из них нам следует использовать? Или, лучше сказать, когда использовать одну, а когда другую? Что ж, скорее всего, услышав об этих шкалах, вы представите себе график какого-нибудь инструмента. Хотя, думая так, вы не ошибаетесь, сама идея и, прежде всего, понятие шкалы не связаны напрямую с финансовым рынком. Шкала помогает нам более адекватно представить значения, которые иначе было бы труднее интерпретировать.

Например, когда мы используем линейку, чтобы что-то измерить, мы, скорее всего, пользуемся арифметической шкалой, поскольку было бы гораздо сложнее понимать и сравнивать разные вещи, если бы значение не изменялось линейно. И именно в этом заключается главное различие между этими двумя шкалами. Логарифмическая шкала не является линейной. Хотя попытка измерить что-либо с помощью логарифмической шкалы может показаться безумием, на самом деле у неё есть множество применений, например для измерения амплитуды сигнала или даже величины силы.

Обратите внимание, что в этих случаях невозможно получить отрицательное значение. Хотя мы можем использовать отрицательные значения в логарифмах, это заставило бы нас работать с комплексными числами. Это лишь усложнило бы всё объяснение и полностью увело бы нас от цели этой статьи. Поэтому давайте ограничимся только простой частью идеи шкалы.

Вкратце: если вам нужно измерять как положительные, так и отрицательные значения и при этом они должны изменяться постепенно и всегда линейно, мы используем арифметическую шкалу. Если данные, которые нужно измерять, растут очень быстро и не могут принимать отрицательные значения, предпочтительнее использовать логарифмическую шкалу. В общем, это всё. Но только практика поможет вам выбрать лучший вариант в каждом конкретном случае.

Хорошо, а что произойдёт, если мы перенесём эту идею в область математики? Что ж, на мой взгляд, это довольно интересная часть. Выражение для арифметической шкалы сводится к следующему.

$$f(x) = \frac{x}{x_{max} - x_{min}}$$

Изображение 01

Возможно, это покажется ерундой. Но обратите внимание на одну вещь: мы можем изменить значение < x >, просто изменив диапазон. В данном случае мы не будем задавать конкретное значение, а приведём

его к определённому диапазону значений. Возможно, на первый взгляд это выражение не имеет особого смысла. Итак, давайте разберём его немного подробнее. Возможно, тогда это станет для вас немного понятнее.

$$(1) \qquad r=\frac{1}{x_{max}-x_{min}}$$

$$(2) \qquad f(x)=x\cdot r$$

Изображение 02

Выражение (1) показывает, насколько мы изменим шкалу. Если значение < r > в выражении (2) меньше единицы, то произойдёт сжатие значений. Если оно больше единицы, то произойдёт растяжение значений. Это похоже на увеличение или уменьшение масштаба внутри системы.

В случае логарифмической шкалы выражение будет немного другим, как вы, вероятно, уже догадываетесь. Дело в том, что для представления данных мы можем использовать любое основание. Однако, как правило, используется основание 10 или основание e, что даёт натуральный логарифм, известный как ln. Возможно, я даже не буду объяснять эту часть логарифмической шкалы, поскольку с точки зрения программирования она очень похожа на то, что мы здесь рассмотрим.

Теперь, чтобы перейти к практической части и придать смысл сказанному, возьмём код из предыдущей статьи и изменим только обучающие данные. Больше мы ничего делать не будем. Важно, чтобы вы увидели и поняли, что это будет единственное изменение. Итак, в исходном коде у нас есть следующий фрагмент:

```
//+------------------------------------------------------------------+
//Выражение для обучения: f(x) = (w0 * 1.8) + (w1 * 2.15) + 2.1
//+------------------------------------------------------------------+
double Train[] {
                  1,    3,  10.35,
                1.5,  3.25, 11.7875,
                1.75, 3.94, 13.721,
                2.85, 3.46, 14.669
               };
//+------------------------------------------------------------------+
#define nColumns    3
#define nLines      Train.Size() / nColumns
//+------------------------------------------------------------------+
```

Фрагмент 01

Этот фрагмент будет заменён на приведённый ниже:

```
//+------------------------------------------------------------------+
//Выражение для обучения: f(x) = (w0 * 1.8) + (w1 * 1.254) + (w2 * 2.35) + 4.1258
//+------------------------------------------------------------------+
double Train[] {
                5.89,   12.473, 18.452,  73.731142,
                8.452,  13.154, 24.2541, 92.831651,
                7.512,  14.576, 15.831,  73.128554,
                3.452,  8.459,  30.1254, 91.741676
               };
//+------------------------------------------------------------------+
#define nColumns    4
```

```
#define nLines      Train.Size() / nColumns
//+------------------------------------------------------------------+
```

Фрагмент 02

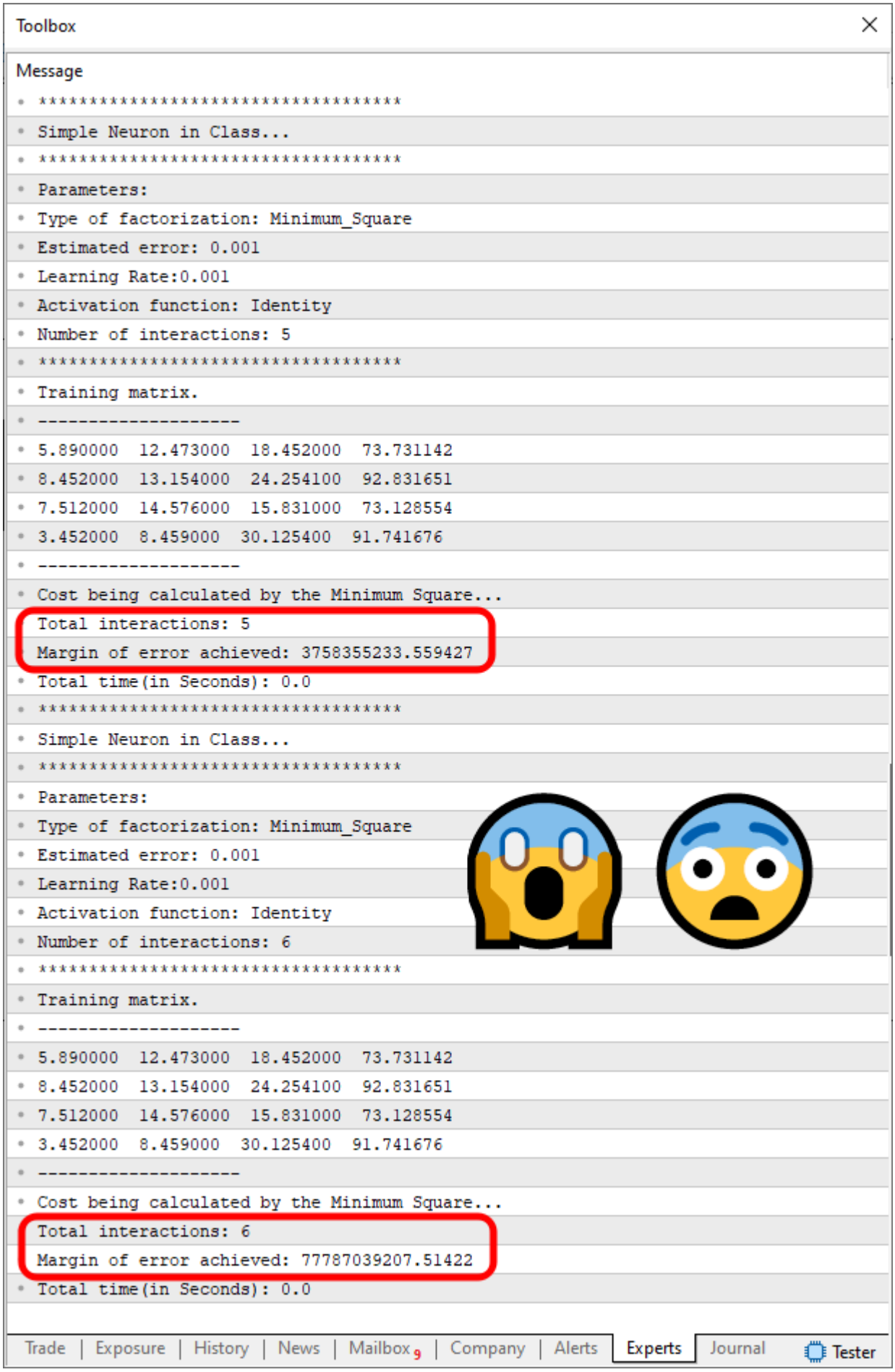
То есть мы изменили только обучающие данные перцептрона. Но есть одна небольшая проблема. Хотя перцептрон может обрабатывать обучающие данные исходного фрагмента, он не сможет сделать то же самое с данными этого второго, изменённого фрагмента. "Как это нет? Вы с ума сошли? Конечно, перцептрон сможет обработать оба фрагмента. Если не сможет, значит, в коде есть какая-то ошибка". Итак, дорогой читатель, прежде чем запустить код, давайте внесём последнее изменение. Таким образом, мы получим полный код, который показан ниже:

```
01. //+------------------------------------------------------------------+
02. #property copyright "Daniel Jose"
03. #property script_show_inputs
04. #property description   "Script to compare execution speed\n" \
05.                         "Here we are using only the CPU.\n" \
06.                         "However, the results are still very interesting."
07. //+------------------------------------------------------------------+
08. #include <Neural Network\C_Check_Neuron.mqh>
09. //+------------------------------------------------------------------+
10. enum eFactorization {
11.                         Minimum_Square,
12.                         Gradient_Descent,
13.                     };
14. //+------------------------------------------------------------------+
15. input eFactorization        user00 = Minimum_Square;        //Тип факторизации
16. input double                user01 = 1e-3;                  //Целевая ошибка
17. input double                user02 = 1e-3;                  //Скорость обучения
18. input C_Neuron::eFnActivate user03 = C_Neuron::Identity;    //Функция активации
19. input ulong                 user04 = 5;                     //Количество итераций
20. //+------------------------------------------------------------------+
21. //Выражение для обучения: f(x) = (w0 * 1.8) + (w1 * 1.254) + (w2 * 2.35) + 4.1258
22. //+------------------------------------------------------------------+
23. double Train[] {
24.                  5.89,       12.473, 18.452,  73.731142,
25.                  8.452,      13.154, 24.2541, 92.831651,
26.                  7.512,      14.576, 15.831,  73.128554,
27.                  3.452,      8.459,  30.1254, 91.741676
28.                };
29. //+------------------------------------------------------------------+
30. #define nColumns    4
31. #define nLines      Train.Size() / nColumns
32. //+------------------------------------------------------------------+
33. void OnStart()
34. {
35.     C_Check_Neuron *neuron;
36.
37.     Print("************************************");
38.     Print("Simple Neuron in Class...");
39.     Print("************************************");
40.     Print("Parameters:");
41.     Print("Type of factorization: ", EnumToString(user00));
42.     Print("Estimated error: ", user01);
43.     Print("Learning Rate:", user02);
44.     Print("Activation function: ", EnumToString(user03));
45.     Print("Number of interactions: ", user04);
```

```
46.      Print("************************************");
47.
48.      neuron = new C_Check_Neuron(nColumns - 1, user03, user00 == Minimum_Square);
49.
50.      (*neuron).View_Training_Data(Train, nColumns);
51.      (*neuron).Performs_Training(Train, user01, user02, user04);
52.      // (*neuron).Check_Training(Train, nColumns);
53.
54.      delete neuron;
55.  }
56.  //+------------------------------------------------------------------+
```

Код 01

Обратите внимание, что, помимо изменённого обучающего массива, у нас есть ещё и строка девятнадцать. Она позволит настроить количество итераций, которое выполнит код. По практическим соображениям строка пятьдесят два, отвечающая за вывод результатов, была закомментирована, поэтому выполняться она не будет. Однако это не так важно, поскольку на самом деле нам нужно, чтобы выполнилась строка пятьдесят один: именно она покажет ошибку перцептрона при таком количестве итераций. А теперь посмотрите, что произойдёт, если мы запустим этот код в терминале MetaTrader 5. Результат показан на изображении ниже.



Изображение 03

Чтобы упростить восприятие, я отметил точное место, где можно увидеть ошибку, которую показывает перцептрон. И, к вашему удивлению, обратите внимание: ошибка не уменьшилась, а, наоборот, сильно выросла. Это означает, что перцептрон не может обрабатывать данные из обучающего массива. Но

почему? Если изменить значения, вы увидите, что перцептрон работает. Но когда мы выходим за определённые пределы, кажется, что он просто немного сходит с ума, полностью теряется и не понимает, куда двигаться.

Это одна из проблем, для решения которой нам придётся изменить шкалу значений. Обратите внимание на то, что я только что сказал. Я не говорю, что вам нужно искать подходящие значения, чтобы их использовать. Я говорю вам, что нужно обмануть перцептрон, заставив его поверить, что он имеет дело с одной вещью, тогда как на самом деле — с другой.

Существует несколько способов сделать это, и у каждого из них своё назначение и вполне конкретная цель. Но здесь, поскольку наша цель — сделать изложение как можно более понятным, мы воспользуемся особенно удобным способом объяснения. Мы сделаем так, чтобы само приложение, которое в данном случае представляет собой скрипт для MetaTrader 5, изменяло значения так, чтобы «глупый» перцептрон не заметил, что на самом деле всё иначе. Для этого перейдём к коду класса C_Check_Neuron.

Одно уточнение: в прикреплённом файле я оставлю исходный код, чтобы вы могли увидеть результаты именно такими, как я показал их на изображении выше, а также код, который мы сейчас реализуем. Так вы сможете на практике понять, как и почему нам нужно делать всё именно так.

Хорошо, ниже показан новый код класса C_Check_Neuron:

```
01. //+------------------------------------------------------------------+
02. #property copyright "Daniel Jose"
03. //+------------------------------------------------------------------+
04. #include <Neural Network\C_Neuron.mqh>
05. //+------------------------------------------------------------------+
06. class C_Check_Neuron : private C_Neuron
07. {
08. //+------------------------------------------------------------------+
09.     private :
10. //+------------------------------------------------------------------+
11.         struct st_0
12.         {
13.             double  Hi,
14.                     Lo,
15.                     Reason;
16.             uint    nColumns,
17.                     nLines;
18.         }m_Limits;
19.         double m_arr[];
20. //+------------------------------------------------------------------+
21.     public  :
22. //+------------------------------------------------------------------+
23.         C_Check_Neuron(const double &arr[], uint nInputs = 1, eFnActivate fn = Ider
24.             :C_Neuron(nInputs, fn, isFx)
25.         {
26.             ZeroMemory(m_Limits);
27.             m_Limits.nColumns = nInputs + 1;
28.             m_Limits.nLines = arr.Size() / m_Limits.nColumns;
29.             m_Limits.Lo = m_Limits.Hi = arr[0];
30.             for (ulong c = 1, m = arr.Size(); c < m; c++)
31.             {
32.                 m_Limits.Hi = MathMax(arr[c], m_Limits.Hi);
33.                 m_Limits.Lo = MathMin(arr[c], m_Limits.Lo);
34.             };
35.             ArrayResize(m_arr, arr.Size());
36.             m_Limits.Reason = 1 / (m_Limits.Hi - m_Limits.Lo);
```

```
37.             for (ulong c = 0, m = arr.Size(); c < m; c++)
38.                 m_arr[c] = arr[c] * m_Limits.Reason;
39.         }
40. //+------------------------------------------------------------------+
41.         ~C_Check_Neuron()
42.         {
43.             ArrayFree(m_arr);
44.         }
45. //+------------------------------------------------------------------+
46.         void View_Training_Data(void)
47.         {
48.             string sz0;
49.
50.             Print("Training matrix.");
51.             Print("--------------------");
52.             for (uchar i = 0; i < m_Limits.nLines; i++)
53.             {
54.                 sz0 = "";
55.                 for (uchar j = 0; j < m_Limits.nColumns; j++)
56.                     sz0 += StringFormat("%f  ", m_arr[(i * m_Limits.nColumns) + j])
57.                 Print(sz0);
58.             }
59.             Print("--------------------");
60.         }
61. //+------------------------------------------------------------------+
62.         void Performs_Training(double epsilon, double LearningRate, ulong limit)
63.         {
64.             ulong it0, it1;
65.
66.             it0 = GetTickCount();
67.             Print("Margin of error achieved: ", Learning(m_arr, epsilon, LearningRa
68.             it1 = GetTickCount();
69.             Print("Total time(in Seconds): ", (it1 - it0) / 1000.0);
70.         }
71. //+------------------------------------------------------------------+
72.         void Check_Training(const double &arr[])
73.         {
74.             string  sz0;
75.             double  mem[];
76.
77.             Print("********** RESULT *************");
78.             ArrayResize(mem, m_Limits.nColumns);
79.             for (uchar i = 0; i < m_Limits.nLines; i++)
80.             {
81.                 sz0 = "";
82.                 for (uchar j = 0; j < m_Limits.nColumns - 1; j++)
83.                     sz0 += StringFormat("%f  ", mem[j] = arr[(i * m_Limits.nColumns
84.                 sz0 += StringFormat("%f", Perceptron(mem));
85.                 Print(sz0);
86.             }
87.             Print("--------------------");
88.             ArrayFree(mem);
89.             View_Variables();
90.         }
91. //+------------------------------------------------------------------+
92. };
93. //+------------------------------------------------------------------+
```

Код 02

Этот код ещё не завершён, и в нём пока есть небольшая проблема. Но мы рассмотрим это позже более подробно. Сначала обратите внимание, что в одиннадцатой строке была добавлена структура. Её цель — создать механизм, который мы будем использовать, чтобы обмануть «глупый» перцептрон. Обратите внимание, что в девятнадцатой строке объявлен массив. А теперь начинается самое интересное. В строке 23, где конструктор теперь принимает массив, нам нужно будет использовать этот новый обучающий массив.

А теперь обратите внимание на то, что мы собираемся сделать. Сначала, в 26-й строке, мы очищаем область памяти, в которой находится структура данных. Далее мы инициализируем некоторые значения этой структуры, а память очищаем для того, чтобы не работать с мусором. Даже если потом мы присвоим значения, всегда стоит убедиться, что всё чисто, потому что можно забыть присвоить какое-нибудь из них, и тогда всё усложнится. А теперь начинается самое интересное. В двадцать девятой строке мы инициализируем максимальное и минимальное значения, а сразу после этого входим в цикл, который пройдёт по всему обучающему массиву в поисках наибольшего и наименьшего значений, содержащихся в нём.

Затем, в 35-й строке, мы выделяем память под внутренний массив класса. В строке 36 мы применяем уравнение, которое уже рассматривали ранее в этой статье, чтобы создать арифметическую шкалу. Так, в строке 37 мы входим в новый цикл. Но на этот раз мы создадим преобразованный обучающий массив на основе исходного обучающего массива. На этом мы завершаем конструктор.

В 41-й строке, где у нас находится деструктор, мы лишь освобождаем выделенную память. Теперь обратите внимание, что остальные процедуры были слегка изменены — ничего такого, на чём стоило бы сейчас особо останавливаться. Давайте теперь посмотрим на код скрипта, приведённый ниже.

```
//+------------------------------------------------------------------+
void OnStart()
{
    C_Check_Neuron *neuron;

    Print("************************************");
    Print("Simple Neuron in Class...");
    Print("************************************");
    Print("Parameters:");
    Print("Type of factorization: ", EnumToString(user00));
    Print("Estimated error: ", user01);
    Print("Learning Rate:", user02);
    Print("Activation function: ", EnumToString(user03));
    Print("Number of interactions: ", user04);
    Print("************************************");

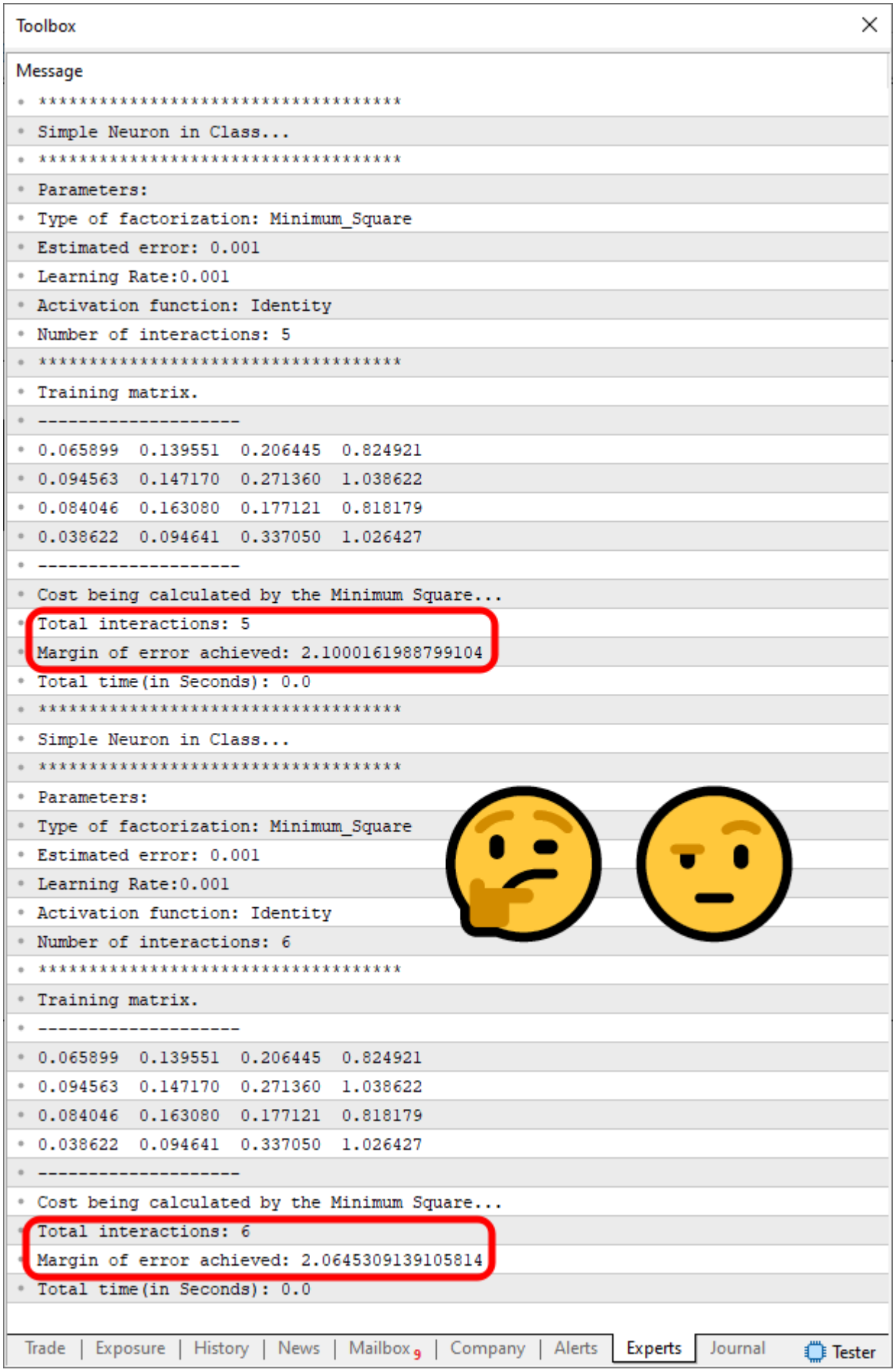
    neuron = new C_Check_Neuron(Train, nColumns - 1, user03, user00 == Minimum_Square);

    (*neuron).View_Training_Data();
    (*neuron).Performs_Training(user01, user02, user04);
    // (*neuron).Check_Training(Train);

    delete neuron;
}
//+------------------------------------------------------------------+
```

Фрагмент 03

Здесь я отмечу только ту часть кода, которая изменилась. Здесь нет ничего, что требовало бы объяснений, учитывая простоту вопроса. Но теперь посмотрите, что происходит, когда мы компилируем и запускаем этот код в терминале MetaTrader 5. Результат показан на изображении ниже.



Изображение 04

"Да что это такое? Не иначе как чудо. Как так получилось, что теперь перцептрон понял ситуацию и начал сходиться? Что за безумие сейчас произошло?" Друг мой, есть вещи, которые можно понять и в которые можно поверить, только когда увидишь их своими глазами. Если бы вы просто об этом

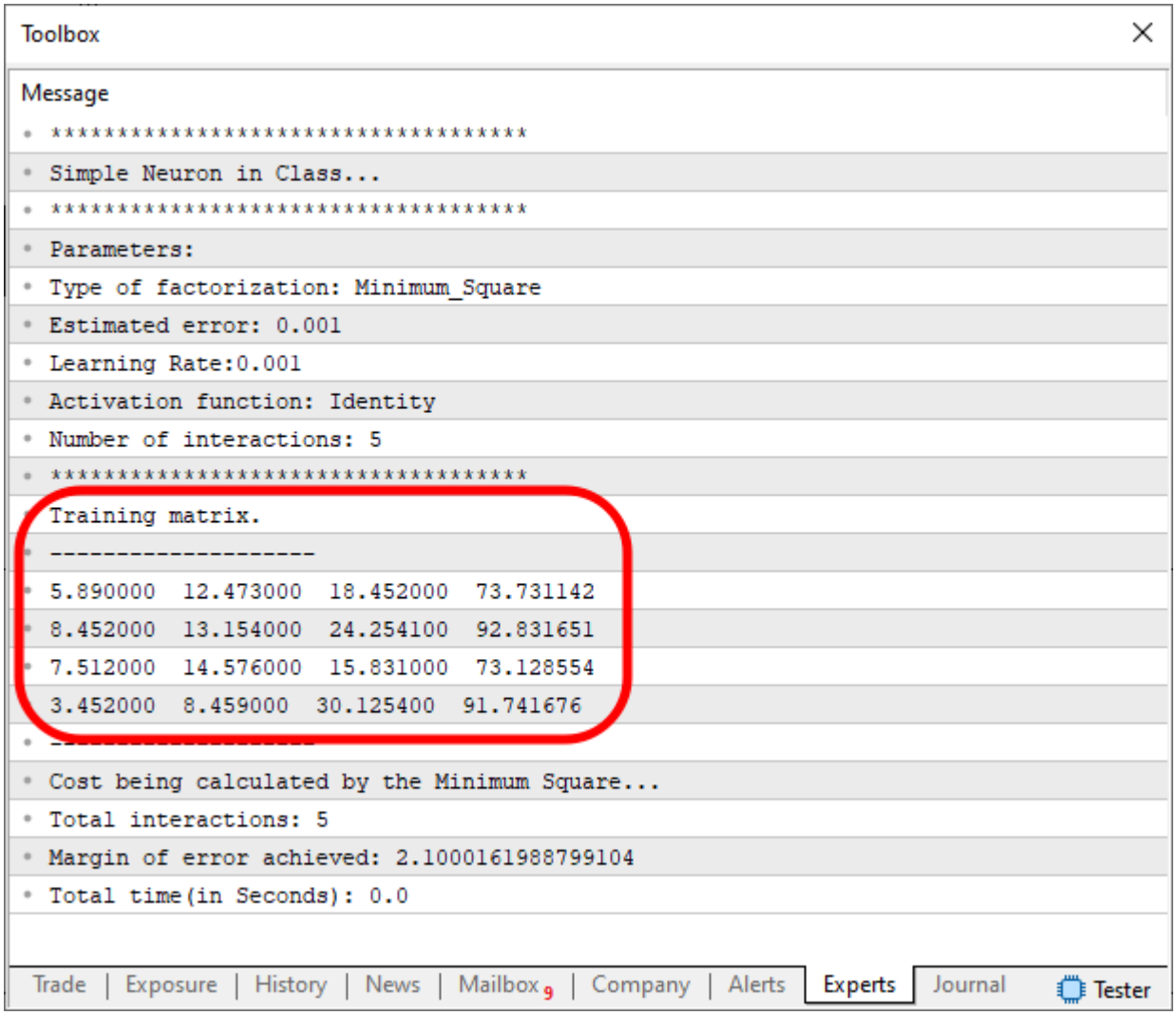
рассказали, никто бы вам не поверил и сказал бы, что вы понятия не имеете, о чём говорите, или, по крайней мере, не понимаете, что делаете. Но вы можете совершенно ясно увидеть, что перцептрон изменил своё поведение. И всё это лишь потому, что мы просто изменили шкалу значений. Ну разве это не безумие? И всё ещё есть люди, которые считают, что в этой жизни уже всё повидали. Я говорю: «*Вы ничего не знаете, наивный*».

Но не всё так просто и не всё так легко принять и понять. Обратите внимание ещё раз на значения, использованные во время обучения. Они отличаются от значения, используемого в коде. Значит, я мухлюю? Ну, не совсем это здесь происходит. Помните, я говорил, что в коде была проблема? Так вот, в этом и проблема. Но повода для беспокойства нет. Всё очень просто. Давайте исправим это, изменив код так, как показано ниже.

```
45. //+------------------------------------------------------------------+
46.          void View_Training_Data(void)
47.          {
48.              string sz0;
49.
50.              Print("Training matrix.");
51.              Print("--------------------");
52.              for (uchar i = 0; i < m_Limits.nLines; i++)
53.              {
54.                  sz0 = "";
55.                  for (uchar j = 0; j < m_Limits.nColumns; j++)
56.                      sz0 += StringFormat("%f  ", m_arr[(i * m_Limits.nColumns) + j]
57.                  Print(sz0);
58.              }
59.              Print("--------------------");
60.          }
61. //+------------------------------------------------------------------+
```

Фрагмент 04

Хорошо, теперь скомпилируем и снова запустим код. Мы получим следующий результат, показанный на изображении ниже:



Изображение 05

Вот теперь да. Помимо того, что мы обманываем этот тупой и глупый перцептрон, мы ещё и создаём у пользователя впечатление, что всё в полном порядке. Но у нас до сих пор есть ещё одна проблема. Чтобы обнаружить эту новую проблему, нужно снова включить код, который показывает результаты. Помимо того, что мы дадим перцептрону сойтись до более подходящего значения ошибки — ведь теперь мы знаем, что он действительно сходится, — мы сможем убрать возможность пользовательской настройки и оставить код в том виде, как показано ниже.

```
//+------------------------------------------------------------------+
void OnStart()
{
    C_Check_Neuron *neuron;

    Print("************************************");
    Print("Simple Neuron in Class...");
    Print("************************************");
    Print("Parameters:");
    Print("Type of factorization: ", EnumToString(user00));
    Print("Estimated error: ", user01);
    Print("Learning Rate:", user02);
    Print("Activation function: ", EnumToString(user03));
    Print("************************************");

    neuron = new C_Check_Neuron(Train, nColumns - 1, user03, user00 == Minimum_Square);

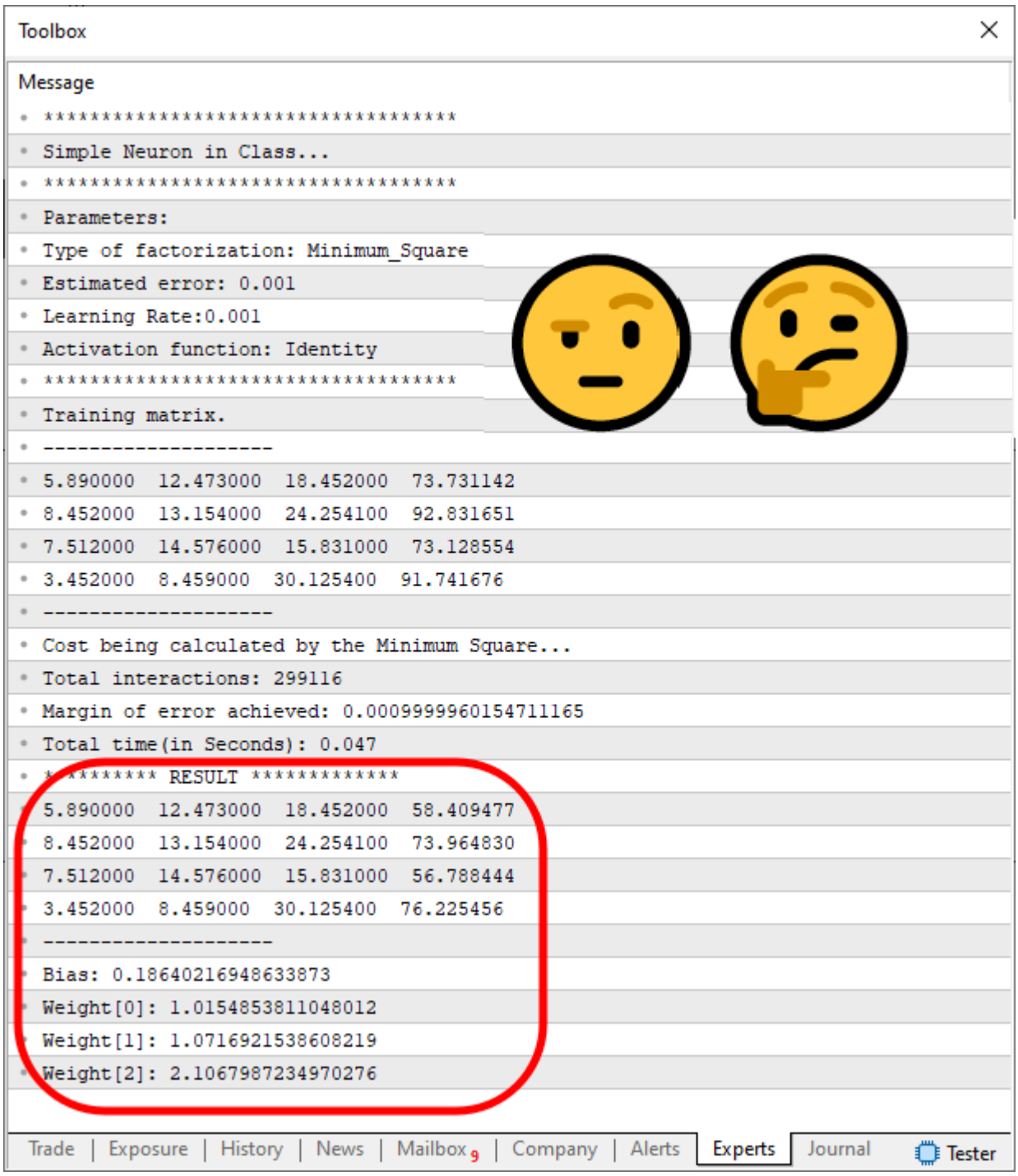
    (*neuron).View_Training_Data();
    (*neuron).Performs_Training(user01, user02, ULONG_MAX);
```

```
      (*neuron).Check_Training(Train);

      delete neuron;
  }
  //+------------------------------------------------------------------+
```

Код 03

И снова компилируем и запускаем новую версию кода, и результат в терминале MetaTrader 5 будет следующим:



Изображение 06

"Но подождите-ка. Это неправильно. Что же произошло, что результат получился настолько плохим?" Что ж, уважаемый читатель, жизнь программиста не всегда идеальна и спокойна. Иногда нам нужно понимать, что мы делаем; иначе в итоге мы получаем ответ, которого сами не понимаем. И, не понимая его, мы в итоге отказываемся от решения, даже когда оно не так уж ошибочно, как может показаться на

первый взгляд. По этой причине мы постараемся понять, что произошло, раз результат так сильно отличался от ожидаемого.

Для этого нам нужно воспользоваться математическим пакетом, в котором можно построить график. Так будет проще понять, что здесь происходит. Я буду использовать приведённый ниже код в SciLab — это бесплатная программа, хотя можно было бы использовать и MatLab. Обе программы дают результат одного и того же типа.

```
clc;
clear;
clf;

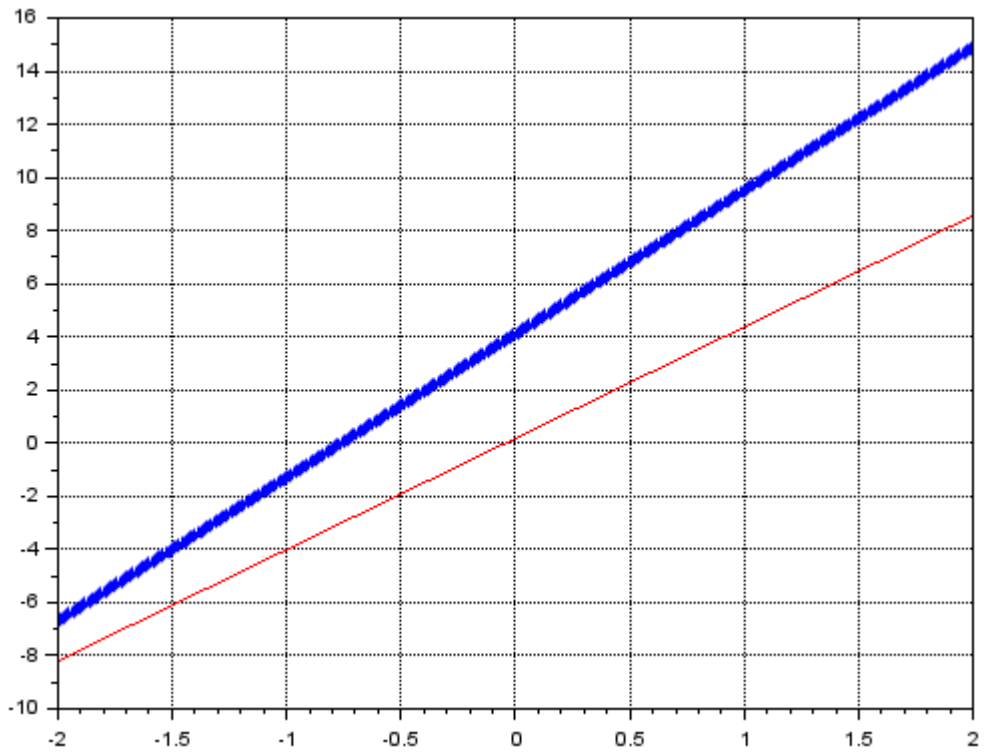
x = [-2:0.01:2];
y =  (1.8 * x) + (1.254 * x) + (2.35 * x) + 4.1258;
plot(x, y, 'p-');

y = (1.0154853811048012 * x) + (1.0716921538608219 * x) + (2.1067987234970276 * x) + 0.
plot(x, y, 'r-');

xgrid(0, 1);
```

Скрипт SciLab

Результат показан ниже.



Изображение 07

Синяя линия представляет исходные данные. Красная линия, в свою очередь, представляет данные, сгенерированные на основе того, чему научился перцептрон. Но есть одна деталь: значение смещения перцептрона неверно. Это связано с тем, что мы изменили шкалу значений. Поэтому нам нужно это исправить. Для этого необходимо изменить следующие фрагменты кода.

```
//+------------------------------------------------------------------+
        inline double Learning(const double &train[], double reason, double epsilon, do
        {
           double err = (m_Infos.IsFx ? Learning_FX(train, epsilon, LearningRate, limit
```

```
            m_Infos.Bias /= reason;

            return err;
        }
 //+------------------------------------------------------------------+
```

Фрагмент 05

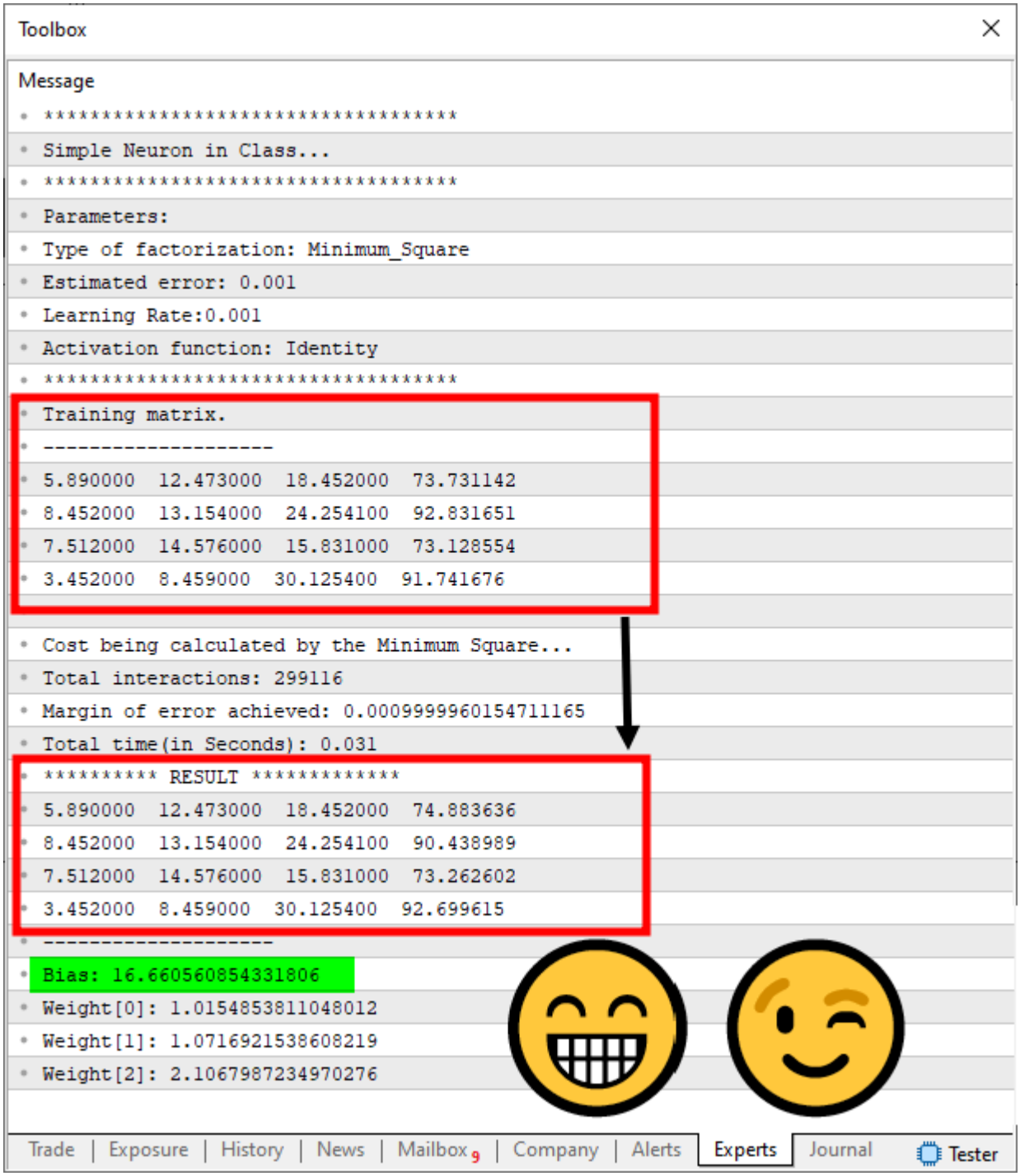
Этот фрагмент исправит проблему смещения и должен быть включён в код класса C_Neuron. После этого нам нужно внести ещё одно изменение, на этот раз в классе C_Check_Neuron, как показано ниже.

```
//+------------------------------------------------------------------+
        void Performs_Training(double epsilon, double LearningRate, ulong limit)
        {
            ulong it0, it1;

            it0 = GetTickCount();
            Print("Margin of error achieved: ", Learning(m_arr, m_Limits.Reason, epsilo
            it1 = GetTickCount();
            Print("Total time(in Seconds): ", (it1 - it0) / 1000.0);
        }
//+------------------------------------------------------------------+
```

Фрагмент 06

Так мы решаем часть проблемы. Теперь снова запустим скрипт и получим следующий результат.



Изображение 08

Обратите внимание, что полученные значения стали значительно ближе к исходным. Но это идеальный вариант? Ну, это зависит от того, зачем мы обучаем перцептрон. Чтобы это проверить, давайте обновим код в SciLab, задав эти новые значения весов и смещения. Тогда код будет выглядеть так, как показано ниже.

```
clc;
clear;
clf;

x = [-2:0.01:2];
y =  (1.8 * x) + (1.254 * x) + (2.35 * x) + 4.1258;
plot(x, y, 'p-');

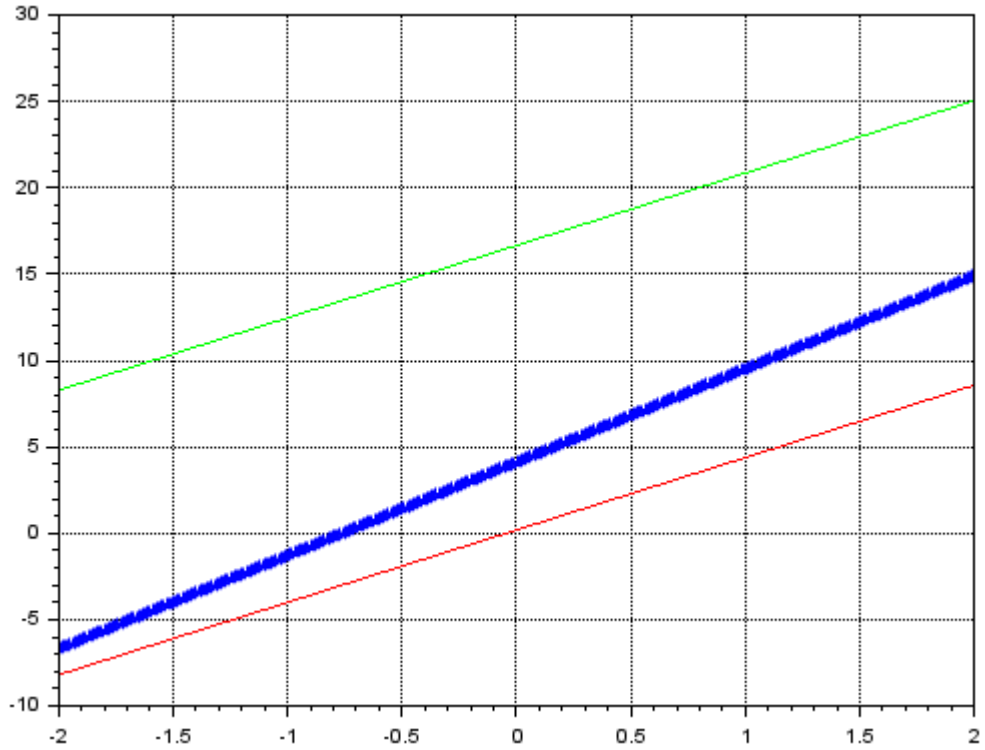
y = (1.0154853811048012 * x) + (1.0716921538608219 * x) + (2.1067987234970276 * x) + 0.
plot(x, y, 'r-');
```

```
y = (1.0154853811048012 * x) + (1.0716921538608219 * x) + (2.1067987234970276 * x) + 16
plot(x, y, 'g-');


xgrid(0, 1);
```

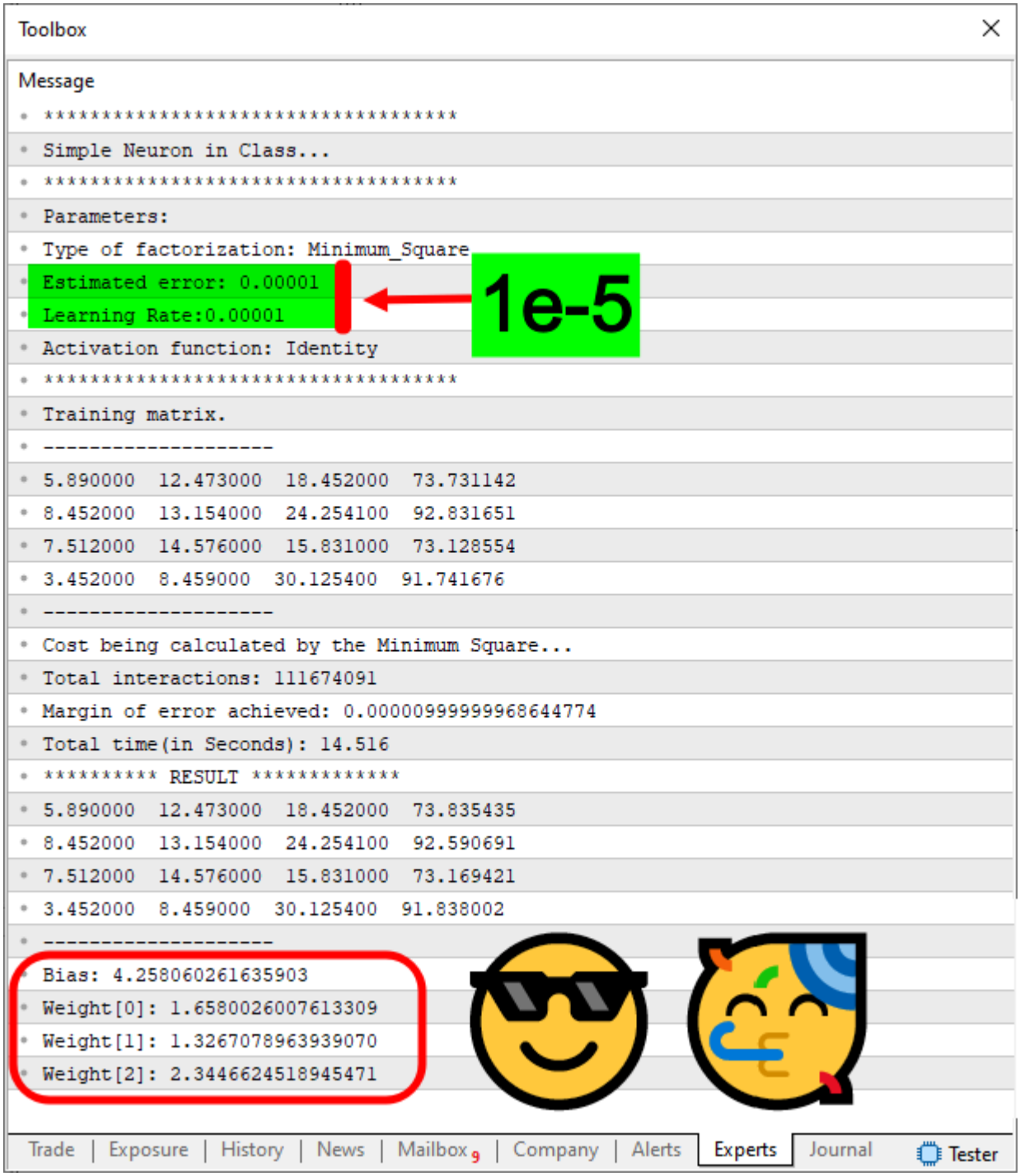
Скрипт SciLab

Мы получим следующий график.



Изображение 09

Обратите внимание, что зелёная линия показывает значения, полученные после корректировки смещения. Хотя эти значения могут подходить для некоторых ситуаций, для многих других они могут оказаться неподходящими. Итак, чтобы попытаться ещё немного улучшить ситуацию, мы изменим значение целевой (ожидаемой) ошибки и скорость обучения. Поскольку значения по умолчанию имеют порядок 1e-3, попробуем использовать значение 1e-5 как для целевой (ожидаемой) ошибки, так и для скорости обучения. Мы получим результат, показанный на изображении ниже:



Изображение 10

Теперь мы сделаем следующее: снова введём эти новые значения весов и смещения в SciLab. Таким образом, мы получим код, показанный ниже.

```
clc;
clear;
clf;

x = [-2:0.25:2];
y =  (1.8 * x) + (1.254 * x) + (2.35 * x) + 4.1258;
plot(x, y, 'p-');

y = (1.0154853811048012 * x) + (1.0716921538608219 * x) + (2.1067987234970276 * x) + 0.
plot(x, y, 'r-');


y = (1.0154853811048012 * x) + (1.0716921538608219 * x) + (2.1067987234970276 * x) + 16
```

```
plot(x, y, 'g-');

y = (1.6580026007613309 * x) + (1.3267078963939070 * x) + (2.3446624518945471 * x) + 4.
plot(x, y, 'mo');

xgrid(0, 1);
```

Скрипт SciLab

При запуске этого нового скрипта мы получим следующий график.

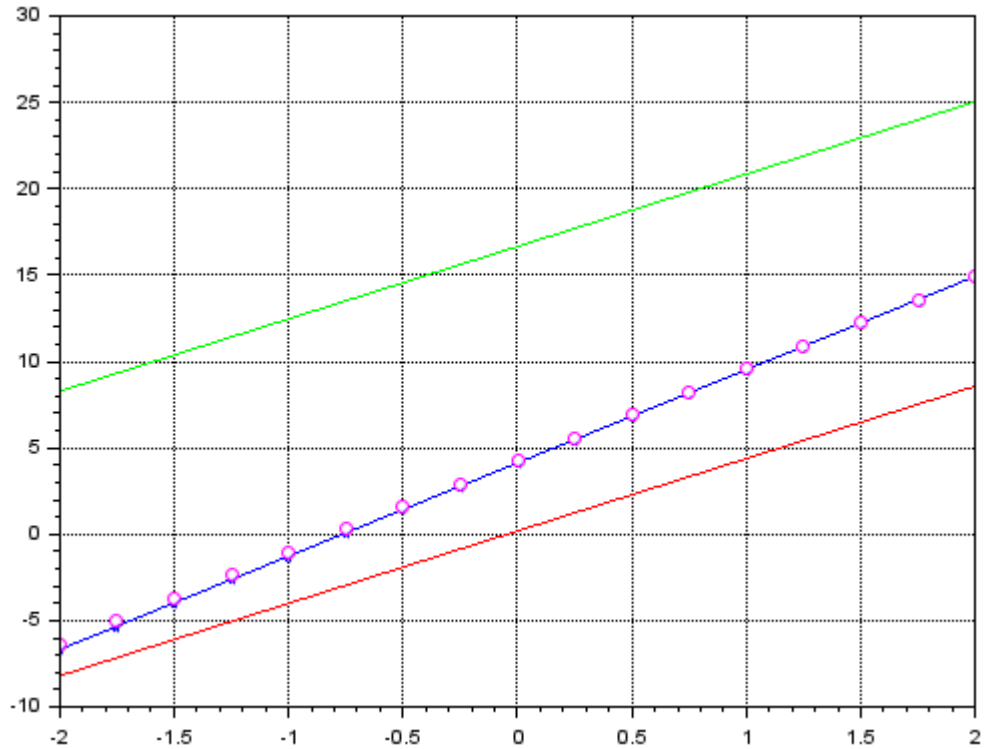


Рисунок 11

Обратите внимание на кое-что интересное. Синяя линия показывает реальные обучающие данные. Зелёная линия показывает значение ошибки, когда мы проводим обучение с целевой (ожидаемой) ошибкой и скоростью обучения 1e-3. Маленькие фиолетовые кружки показывают результат, когда мы используем тот же код, но со значением 1e-5. Такой результат действительно удивляет.

Заключительные замечания

В этой статье я показал, что нам ещё многое предстоит изучить, прежде чем думать, будто мы уже всё знаем. Я уже не раз упоминал, что, несмотря на все достижения, ЧЕЛОВЕЧЕСТВО ДО СИХ ПОР НЕ ЗНАЕТ, КАК СОЗДАТЬ ОКОНЧАТЕЛЬНУЮ СЕТЬ ПЕРЦЕПТРОНОВ. Мы проводим исследования и пытаемся учиться на собственных успехах и ошибках.

Я здесь не для того, чтобы кого-то очернять или оскорблять, и мне даже не нужно ничего никому доказывать. Я здесь просто для того, чтобы поделиться своим опытом как программиста. И, как стало ясно из этой статьи, недостаточно просто собрать набор данных, чтобы создать что-то, действительно способное имитировать человеческий разум.

Есть детали, о которых многие не знают. Но после прочтения этой статьи вы уже понимаете, что нам часто приходится настраивать код, чтобы превратить на первый взгляд разрозненные данные в нечто действительно пригодное для использования.

В прикреплённом файле вы найдёте итоговый код, а также код, использованный в начале этой статьи. Не забудьте изучить предыдущие статьи, потому что нам ещё предстоит долгий путь.

Файл MQ5	Описание
Scripts\Example A	Базовая демонстрация

Следующая статья >>

Daniel Jose

Бразилия 52271

Другие статьи автора

- Нейронная сеть на практике: Ручное обратное распространение
- Нейронная сеть на практике: Начинаем строить цепочку
- Нейронная сеть на практике: Перцептрон
- Нейронная сеть на практике: Градиент
- Нейронная сеть на практике: Функции активации
- Нейронная сеть на практике: График функции ReLU

Подпишитесь и узнавайте о популярных статьях и кодах